

Batched Paillier-Based Hamming-Distance Computation over Binary Embeddings

Yavor Litchev
ylitchev@stanford.edu

Liwen Ouyang
liwen@xtrace.ai

Abstract

Additively homomorphic encryption supports outsourced computation on encrypted binary embeddings, but large-integer arithmetic and data movement can limit throughput. We describe a Paillier-based client that combines a carry-separated binary encoding, table-based encryption, reduced-exponent decryption, CUDA/CGBN arithmetic, persistent device state, and batched retrieval integration. We establish the encoding’s correctness and characterize four CPU and GPU client configurations. The lookup configuration uses a 280-bit exponent-size parameter. Across 3 warm-state trials on batches of 10,000 random 512-bit embeddings, the lookup GPU configuration achieved median-batch throughputs of 43,091 encryptions/s and 28,983 Hamming-distance decodes/s. Its amortized costs were 0.0232 ms and 0.0345 ms per vector, corresponding to factors of 453.8 and 200.9 relative to the measured CPU baseline. These implementation-specific results demonstrate the throughput benefits of combining cryptographic precomputation, batched accelerator execution, and persistent runtime state. The study distinguishes warm-batch performance from isolated-request latency and identifies the remaining costs of initialization, transport, and retrieval integration.

Keywords: Paillier encryption, homomorphic encryption, Hamming distance, GPU big-integer arithmetic, binary embeddings.

1 Introduction

Binary embeddings support similarity search through Hamming distance, which counts the coordinates at which two vectors differ. When embeddings must remain confidential to a computation service, additive homomorphic encryption provides a means of evaluating a packed representation without first decrypting it. Paillier encryption [1] is relevant to this setting because ciphertext multiplication implements plaintext addition. Its cost nevertheless includes large-integer arithmetic, randomized encryption, and the movement of expanded ciphertext representations.

This paper examines the implementation of that arithmetic pipeline in the XTrace SDK. The client packs binary coordinates into separated bit positions, encrypts the packed values, and decodes homomorphically combined ciphertexts into Hamming distances. The implementation includes CPU and GPU backends for both conventional Paillier and a lookup-based configuration. The latter changes the generator, randomization procedure, and decryption exponent in addition to using precomputed tables.

The engineering objective is to reduce repeated work across the complete client call. Message tables replace exponentiation by modular products, cooperative GPU arithmetic distributes each fixed-width integer across a group of threads, and persistent allocations amortize table construction and transfer. The surrounding retrieval code encrypts each query once, decodes returned results in batches, and fetches document contents only after local ranking.

The performance measurements show substantial differences among these implementations. The evaluation fixes the lookup exponent-size parameter at 280 bits and measures large warm batches rather than end-to-end search. The resulting ratios characterize the complete client implementations, including their arithmetic and representation choices. They do not isolate the contribution of each optimization or establish a comparison against the fastest available Paillier implementation.

The study provides the following:

- A precise description of the packed Hamming-distance computation, including its carry and plaintext-range conditions.
- An account of table-based encryption, cooperative GPU arithmetic, device residency, and retrieval integration.
- Measurements of four client configurations, with explicit parameter settings, timing boundaries, and limits on causal interpretation.

Implementation Availability. An implementation of the Paillier clients is publicly available in the XTrace SDK repository at <https://github.com/XTraceAI/xtrace-sdk>. The repository provides installation instructions, usage examples, and tests for readers who wish to evaluate the implementation.

2 Background and Scope

2.1 Cryptographic and Implementation Context

Paillier’s constructions are based on composite residuosity [1]. The earlier Goldwasser–Micali scheme provides probabilistic encryption [2], and Damgård–Jurik extends Paillier to larger plaintext spaces [3]. These schemes differ in their algebra and ciphertext expansion, so their suitability depends on the computation being implemented.

Precomputation is also an established implementation technique. For example, Rache constructs encryptions using cached ciphertexts and radix decompositions [4]. Our message table similarly exploits a fixed radix, but this observation does not establish the security of our randomization procedure or imply equivalence between the constructions. On the arithmetic side, CGBN provides cooperative fixed-width GPU big-integer operations [5]. The present contribution is the integration and characterization of these techniques for a particular client workload, not a claim that lookup tables or GPU big-integer arithmetic are new.

2.2 System Model and Information Exposure

Let $x, y \in \{0, 1\}^d$ be binary embeddings. The desired similarity statistic is the Hamming distance

$$\text{Ham}(x, y) = \sum_{i=0}^{d-1} \mathbf{1}[x_i \neq y_i].$$

The client holds the secret key, and the server combines encrypted query and data vectors under the same public key. The intended confidentiality goal concerns an honest-but-curious server that follows the arithmetic protocol but may inspect its inputs and outputs. We assume that the key-holding client and its device are trusted. Access patterns, record identifiers, candidate counts, and metadata supplied as filters are not hidden.

The returned ciphertexts encrypt packed coordinate sums, not merely the scalar distance. A secret-key holder can recover these sums and, given its query, reconstruct the corresponding data vector. Consequently, the protocol does not provide distance-only disclosure to a mutually untrusted querying client. It also does not establish malicious-server correctness, chosen-ciphertext security, or side-channel resistance. Ranking occurs after decryption at the client, rather than through encrypted top- k selection.

3 Packed Hamming-Distance Computation

3.1 Conventional Paillier Baseline

Let $n = pq$ for distinct primes p and q . Conventional Paillier encrypts $m \in \mathbb{Z}_n$ using a suitable public generator g and a fresh random $r \in \mathbb{Z}_n^*$:

$$\text{Enc}(m, r) = g^m r^n \bmod n^2. \quad (1)$$

Its additive homomorphism gives

$$\text{Dec}(\text{Enc}(m_1, r_1) \text{Enc}(m_2, r_2) \bmod n^2) = (m_1 + m_2) \bmod n.$$

Our conventional CPU baseline uses $g = 1 + n$ and decrypts using $\phi(n) = (p - 1)(q - 1)$ and its inverse modulo n . Although the source evaluates g^m with a general modular-exponentiation routine, the binomial theorem gives

$$(1 + n)^m \equiv 1 + mn \pmod{n^2}. \quad (2)$$

Thus a message exponentiation is not inherently necessary for this baseline generator. This distinction matters when interpreting the reported encryption ratios.

3.2 Encoding and Correctness

For a chunk of s coordinates, define

$$E_s(x) = \sum_{i=0}^{s-1} x_i 4^{s-1-i}, \quad M_s = \sum_{i=0}^{s-1} 4^i. \quad (3)$$

The big-endian binary representation of $E_s(x)$ is

$$0, x_0, 0, x_1, \dots, 0, x_{s-1}.$$

For binary inputs, each base-4 digit of $E_s(x) + E_s(y)$ belongs to $\{0, 1, 2\}$. No digit therefore carries into its neighbor. The low bit of each digit is one exactly when its two input bits differ. Provided the plaintext sum does not wrap modulo n ,

$$\text{Ham}(x, y) = \text{popcount}((E_s(x) + E_s(y)) \& M_s), \quad (4)$$

where $\&$ denotes bitwise AND. These low bits occupy odd zero-based positions when the padded string is read from the left, but even bit offsets when indexed from the least significant bit. Specifying the convention avoids ambiguity in the decode mask.

A sufficient range condition for every pair of s -bit chunks is

$$\frac{2(4^s - 1)}{3} < n. \quad (5)$$

This is a correctness requirement, not merely a performance consideration. For longer vectors, the construction applies independently to chunks satisfying this condition and adds their decoded counts. The measured workload has $s = d = 512$ and uses one ciphertext per vector. The implementation parameter `key_len=1024` specifies the bit length of each prime, not of n . The resulting modulus has 2047 or 2048 bits, comfortably satisfying Equation (5) for this workload. No claim about arbitrary chunk sizes follows from this one-chunk experiment.

4 Lookup Construction and Precomputation

4.1 Reduced-Exponent Decryption

The lookup client uses a Paillier-based subgroup construction rather than simply attaching tables to the baseline. Key generation chooses primes $q_1 \mid (p - 1)$ and $q_2 \mid (q - 1)$, and sets

$$a = \text{lcm}(q_1, q_2).$$

Generators of orders q_1 and q_2 modulo p and q are lifted to the squared prime moduli and combined by the Chinese remainder theorem. The intended construction has $\text{ord}_{n^2}(g) = na$, $\text{ord}_n(g) = a$, and

$$g^a \equiv 1 + n\beta \pmod{n^2}, \quad \gcd(\beta, n) = 1. \quad (6)$$

The last condition is necessary for the decryption inverse to exist.

Writing $h = g^n \text{ mod } n^2$, the lookup encryption has the form

$$c = g^m h^\rho \text{ mod } n^2, \quad (7)$$

where the implemented table sampling determines the effective exponent ρ . Since $h^a = 1$, exponentiation gives

$$c^a \equiv (g^a)^m \equiv 1 + nm\beta \pmod{n^2}.$$

For $u \equiv 1 \pmod{n}$, define $L(u) = (u - 1)/n$. Decryption therefore recovers

$$m = L(c^a \text{ mod } n^2) \beta^{-1} \text{ mod } n. \quad (8)$$

The client precomputes β^{-1} . This derivation establishes the algebraic identity for valid ciphertexts and keys. The reduced exponent lowers the number of wide modular multiplications required during decryption.

4.2 Parameter Selection

The lookup configuration uses `alpha_len=280`, matching the native CUDA extension’s default `ALPHA_LEN`. Key generation divides this parameter between two 140-bit subgroup primes q_1 and q_2 and sets $a = \text{lcm}(q_1, q_2)$. For distinct subgroup primes, the resulting exponent has 279 or 280 bits, and the benchmark records its actual bit length. The parameter specifies the target size of the decryption exponent, not the modulus size or a numerical security level.

The CPU and GPU lookup configurations use this same parameter setting. It should be supplied explicitly when reproducing the study, since defaults at the Python and native interfaces need not coincide. The present paper evaluates implementation performance at these fixed parameters. It does not derive a new security proof or infer cryptographic security from timing measurements.

4.3 Message Lookup Table

For the nonstandard generator in Equation (7), the client decomposes a plaintext into base-256 digits:

$$m = \sum_{i=0}^{b-1} d_i 2^{8i}, \quad d_i \in \{0, \dots, 255\}.$$

It precomputes $T_{i,j} = g^{j2^{8i}} \bmod n^2$ for each digit position and byte value. Online encryption then evaluates

$$g^m \equiv \prod_{i=0}^{b-1} T_{i,d_i} \pmod{n^2}.$$

The table depends only on public parameters and can be shared between compatible contexts. At the configured capacity of 2048 plaintext bits, $b = 256$. With 4096-bit fixed-width entries, the 256×256 message table occupies 32 MiB before allocator or host-object overhead. This is a representation-size calculation, not a measured process-memory result.

4.4 Precomputed Randomized Masking

The random factor is also precomputed. The client constructs 256 entries

$$\eta_j = h^{r_j} \bmod n^2$$

and multiplies 14 independently selected entries, with replacement, for each encryption. The resulting factor is $h^{\sum_{t=1}^{14} r_{J_t}}$, so it preserves the algebraic decryption identity. Noise tables are retained per client instance rather than shared through the deterministic message-table cache.

Noise-table state is distinct from the deterministic public message table. The implementation maintains this distinction in its caching policy and samples fresh indices for each ciphertext. Analysis of the resulting randomization distribution and its reuse is separate from the algebraic correctness and performance evaluation presented here.

5 GPU Arithmetic Implementation

5.1 Cooperative Fixed-Width Arithmetic

The GPU implementation uses CUDA and CGBN [5]. Each CGBN environment has a fixed integer width, with a cooperative group operating on one big-integer instance. The lookup extension’s default configuration assigns 32 threads per instance and reserves 4096 bits for residues modulo n^2 when `KEY_BITS=1024`. Wider intermediate products support modular reduction without truncating the arithmetic result.

Independent ciphertext chunks supply parallel work across cooperative groups. Table-based encryption consists primarily of indexed loads and modular products, while decryption performs exponentiation followed by the L map and multiplication by a precomputed inverse. Batching amortizes kernel-launch and host/device transfer costs. The measurements in Section 7 characterize a batch of 10,000 vectors and do not establish a crossover point at which GPU execution becomes preferable.

5.2 Combination, Decode, and Re-encryption

Homomorphic combination is pointwise ciphertext multiplication modulo n^2 . After decryption, the GPU decode path applies the alternating-bit mask in Equation (4) and counts the selected bits. Keeping this operation on the device avoids returning full plaintext chunks merely to compute a scalar count. The server multiplication and client decoding are distinct operations, and only the latter is included in the decode measurement.

The lookup extension also implements a fused decrypt-then-re-encrypt kernel. This operation retains intermediate plaintext on the device and encrypts it under a target key, provided the plaintext fits the target modulus. It requires access to the source secret key and is therefore a trusted re-encryption operation, not a proxy re-encryption protocol. It is not included in the four-configuration evaluation, and no quantitative speedup is claimed for it here.

6 Runtime and Retrieval-System Optimizations

6.1 Lazy Initialization and Persistent Tables

The lookup GPU backend separates context metadata loading from expensive runtime preparation. Configuration loading can defer table construction and device allocation until the tables are needed. A process-wide host cache reuses deterministic message tables for compatible public parameters. Lookup and noise tables remain resident on the GPU across calls, subject to the lifetime of the client context. The CPU lookup path need not follow the same initialization policy. Warm-state measurements therefore exclude costs that remain relevant to newly created contexts and short-lived applications.

6.2 Raw-Byte Interchange

The integration supports ciphertexts represented as little-endian raw bytes. JSON requests encode these bytes using base64, while MessagePack responses can carry binary values without that textual expansion. The Python client accepts byte-valued ciphertexts and normalizes them as required by the selected backend.

Representation is not uniform across all implementations. Native bindings and compatibility paths also expose Python integers, GMP-backed objects, and string conversions. In particular, the conventional GPU wrapper retains hexadecimal conversion in its decode interface. Byte-oriented transport should therefore not be conflated with a claim that every measured Python/C++ call is conversion-free or zero-copy. The reported public-method timings include the conversions performed by each measured backend.

6.3 Retrieval Orchestration

The SDK retriever converts a query embedding to binary form, encrypts it once, and submits it with an execution-context identifier. Optional metadata and range filters are passed in the same request to restrict candidate evaluation. Returned encrypted results are associated with record identifiers. By default, the retriever decodes them through `decode_hamming_client_batch`.

An explicitly selected multiprocessing mode instead dispatches individual decode calls through a process pool. This is an alternative execution mode, not an automatic fallback or an additional layer of GPU parallelism. Its benefit depends on process startup, serialization, and backend compatibility. Ranking uses NumPy sorting, and content retrieval and AES decryption are deferred until the top- k identifiers have been selected. These observations concern SDK orchestration of server requests. They

do not demonstrate server-internal parallelism, and neither server scalability nor multiprocessing speedup is measured here.

7 Evaluation

7.1 Methodology

We benchmarked the public batch methods for vector encryption and Hamming-distance decoding:

`encrypt_vec_batch` and `decode_hamming_client_batch`.

The benchmark used $N = 10,000$ random binary embeddings of dimension $d = 512$ and one random binary query. The input generator used seed 1337. Each of four configurations constructed a separate key pair with `key_len=1024`. Both lookup clients were explicitly configured with `alpha_len=280`, corresponding to Section 4.2. The harness verified the selected CPU or GPU backend and the runtime exponent-size parameter before measurement. The configurations are conventional Paillier CPU, lookup CPU, conventional Paillier GPU, and lookup GPU. The term “lookup” denotes the complete construction in Section 4, not an isolated table toggle.

Each client first processed a warmup batch of 128 vectors through encryption and distance decoding. The experiment then measured 3 encryption calls followed by 3 decode calls per configuration, using the same input batch and key within that configuration. We report the median batch time for each method. The harness measured wall-clock time with Python’s `time.perf_counter`, with garbage collection before each timed call. Calls returned host-visible results, so these measurements include the corresponding wrapper work and device transfers, rather than only CUDA kernel time. The CPU batch methods process vectors serially. The optional retriever process pool was not used.

Key generation, initialization, warmup, query encryption, and construction of the combined ciphertexts were outside the timed regions. After the encryption trials, the harness formed the decode batch from the final encryption output by multiplying each ciphertext with an encrypted query modulo n^2 . This batch was reused across the decode trials. The harness did not contact a server. All 10,000 returned distances in every decode trial matched the plaintext reference, for 120,000 checked results across the four configurations. These checks establish agreement on the sampled workload, not exhaustive correctness or cryptographic security.

Measurements were collected on an AMD Ryzen 7 5800X CPU and an NVIDIA GeForce RTX 3080 with 10 GB of memory, using Python 3.12.3, gmpy2 2.3.0, GMP 6.3.0, and NVIDIA driver 580.173.02. The archived record includes individual timings, runtime configurations, the SDK source revision, installed toolchain information, and hashes of the relevant sources and native extensions. The experiment used a shared workstation without exclusive GPU access. Three trials on one key per configuration support descriptive comparisons, not confidence intervals or estimates of variation across independently generated keys.

7.2 Results

For batch time T in seconds, the amortized cost is $1000T/N$ milliseconds per vector and throughput is N/T vectors per second. The former is not the response latency of an isolated vector or a complete search request.

Table 1 reports the normalized costs and throughputs, and Figure 1 reports the ratios. Relative to the measured CPU baseline, lookup CPU is $16.1\times$ faster for encryption and $6.5\times$ faster for decode. Conventional Paillier GPU has ratios of $39.7\times$ and $31.0\times$, respectively. Lookup GPU has the largest ratios, $453.8\times$ and $200.9\times$. Within the lookup family, the GPU implementation is $28.2\times$

Client	Amortized cost (ms/vector)		Throughput (vectors/s)	
	Encrypt	Decode	Encrypt	Decode
Paillier CPU	10.5320	6.9328	94.9	144.2
Lookup CPU	0.6547	1.0678	1,527.3	936.5
Paillier GPU	0.2652	0.2235	3,770.0	4,474.0
Lookup GPU	0.0232	0.0345	43,090.8	28,983.5

Table 1: Warm-batch measurements for 10,000 random 512-bit embeddings, computed from the median of 3 trials per method and configuration. Both lookup clients use `alpha_len=280`. Costs are amortized per vector.

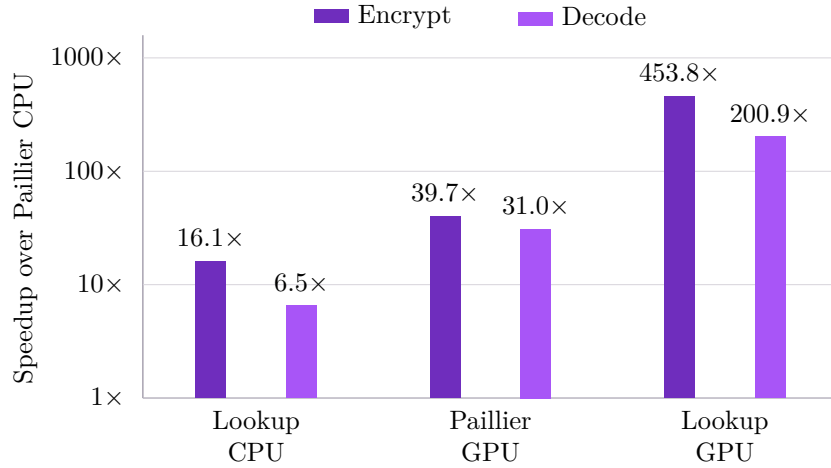


Figure 1: Speedup relative to the measured Paillier CPU implementation on a logarithmic axis, using ratios of median batch times. Ratios compare complete client configurations. The lookup variants combine table-based encryption and reduced-exponent decryption, so the comparison is not an isolated lookup-table ablation.

faster for encryption and 30.9 $\times$ faster for decode than its CPU counterpart. Appendix A reports the observed timing ranges.

7.3 Interpretation and Experimental Limits

The CPU/GPU comparisons within each construction provide evidence that batched accelerator execution benefits this implementation. They do not isolate GPU arithmetic from differences in wrappers, serialization, or decoding. Likewise, lookup/non-lookup comparisons jointly change the generator, exponent length, randomization, and table usage. In particular, faster lookup decoding cannot be attributed to the message table, which is not used by the decryption formula. The experiment is not an ablation study of individual optimizations.

The conventional CPU reference also leaves Equation (2) unused and does not provide a tuned multicore or CRT-based decryption baseline. The headline ratios are therefore relative to this specific reference, not to the fastest available Paillier software. A broader evaluation would add tuned CPU baselines, repeated measurements across multiple keys, and experiments varying batch size, embedding dimension, and hardware. Cold-start cost, peak memory, network transfer, and full retrieval latency would require separate measurements.

8 Explored Alternatives

During development, we explored CPU vectorization through Intel IPP Cryptography and the `ippcp.h` interface. Intel’s Cryptography Primitives library supports several SIMD instruction sets, including AVX-family extensions [6]. Our prototype did not outperform the GMP-based path. Data rearrangement, carry handling, and movement between vector operations are possible explanations, but we do not report profiling measurements that isolate these mechanisms. This observation is not a general conclusion about AVX or Intel’s library.

We also implemented clients for Goldwasser–Micali and Damgård–Jurik encryption [2, 3]. Neither prototype provided higher performance than our Paillier implementation in the development workloads we tried. These exploratory comparisons are not accompanied by a controlled benchmark record in this paper. They explain the engineering direction taken, but do not establish superiority over those schemes at matched security or across other workloads.

9 Limitations and Future Work

The evaluation concerns client-side computation at fixed cryptographic parameters. It does not measure complete retrieval latency or establish guarantees beyond the system model in Section 2. Metadata leakage, disclosure of packed sums to the key-holding client, and result integrity remain separate protocol considerations.

The systems techniques have separate limitations. Fixed-width kernels constrain supported parameters, precomputation consumes memory, and persistent tables favor long-lived contexts. Small batches and cold starts may have different performance characteristics. The current evaluation does not quantify these tradeoffs or the benefit of fused re-encryption and retrieval parallelism.

Operational experience motivated attention to ciphertext transmission after local arithmetic became faster. With a 4096-bit ciphertext representation, 10,000 one-chunk results occupy 5.12 MB before identifiers and protocol framing. This is a payload-size calculation, not a measured network bottleneck in the experiment. Byte-oriented transport can remove textual overhead but does not remove the cryptosystem’s ciphertext expansion. Future work should measure end-to-end transfer costs and investigate candidate pruning, request batching, and protocols that return fewer encrypted values. Reducing protocol traffic is a more defensible objective than assuming that pseudorandom ciphertext bytes admit substantial general-purpose compression.

10 Conclusion

We described a batched Paillier-based Hamming-distance client and the interactions among packing, precomputation, GPU arithmetic, and runtime state. Both GPU configurations outperformed their CPU counterparts on the evaluated batch. With the 280-bit exponent-size parameter, the lookup GPU configuration achieved 43,091 encryptions/s and 28,983 Hamming-distance decodes/s, based on median batch times. Relative to the measured conventional CPU baseline, the corresponding speedups were $453.8\times$ and $200.9\times$. These results characterize the combined implementation rather than individual optimizations in isolation. Further evaluation should extend the comparison to tuned CPU baselines, additional workloads, and complete retrieval measurements, with particular attention to ciphertext transport.

A Timing Ranges and Reproducibility

Table 2 reports the minimum and maximum amortized costs observed in the 3 trials for each method. These ranges describe variability within this run and are not confidence intervals. The ancillary materials contain the raw JSON record, a snapshot of the benchmark harness, and the script that generates the numerical values used throughout the manuscript. The harness requires the corresponding SDK and native extensions. Artifact hashes identify the measured files but do not replace the implementation dependencies needed for a complete reproduction.

Client	Encrypt range (ms/vector)	Decode range (ms/vector)
Paillier CPU	10.4244–10.5369	6.9221–6.9472
Lookup CPU	0.6460–0.6564	1.0673–1.0759
Paillier GPU	0.2649–0.2702	0.2235–0.2316
Lookup GPU	0.0231–0.0235	0.0344–0.0351

Table 2: Observed minimum and maximum amortized costs across 3 warm-batch trials per method and configuration.

References

- [1] P. Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology, EUROCRYPT 1999*, LNCS 1592, pp. 223–238. Springer, 1999. [doi:10.1007/3-540-48910-X_16](https://doi.org/10.1007/3-540-48910-X_16).
- [2] S. Goldwasser and S. Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984. [doi:10.1016/0022-0000\(84\)90070-9](https://doi.org/10.1016/0022-0000(84)90070-9).
- [3] I. Damgård and M. Jurik. A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system. In *Public Key Cryptography, PKC 2001*, pp. 119–136. Springer, 2001. [doi:10.1007/3-540-44586-2_9](https://doi.org/10.1007/3-540-44586-2_9).
- [4] D. Zhao. Rache: Radix-additive caching for homomorphic encryption. arXiv:2201.04255, 2022. <https://arxiv.org/abs/2201.04255>.
- [5] NVIDIA Research. CGBN: CUDA accelerated multiple precision arithmetic using cooperative groups. <https://github.com/NVlabs/CGBN>. Accessed September 15, 2026.
- [6] Intel. Intel Cryptography Primitives Library. <https://github.com/intel/cryptography-primitives>. Accessed September 15, 2026.